

بناام خدا

امتحان پایانترم درس مبانی کامپیوتر و برنامهسازی

نام و نام خانوادگی: تاریخ: ۲۸ دی ۱۳۹۶ شماره دانشجویی: مدت امتحان: ۵.۲

ساعت

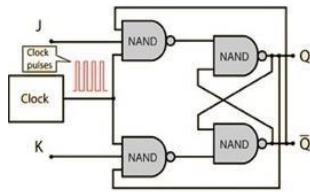
نام استاد:

نکات:

- به هیچ سوالی در طول این امتحان پاسخ داده نمی شود. اگر سوالی ابهام دارد و یا ایرادی در آن می بینید فرضیات لازم را بهعمل آورید .
- لطفاً نام خانوادگی و شماره دانشجویی خود را روی تمام صفحات بنویسید.
- استفاده از کتاب، جزوه و هر وسیله ای بجز خودکار یا مداد ممنوع است.
- پاسخ هر سوال را در فضای تعیین شده بنویسید. اگر نیاز به فضای بیشتر دارید از پشت همان صفحه استفاده نمایید.
- موبایل خود را خاموش کرده و در کیف بگذارید. در دسترس بودن موبایل معادل صفر در امتحان است.
- مدت امتحان دو ساعت میباشد.

سوال	نمره
۱	۶/
۲	۱۵/
۳	۵/
۴	۱۲/
۵	۸/
۶	۱۸/
۷	۳۲/
۸	۶/
مجموع	/۱۰۰

سوال ۱- (۶ نمره)



الف (۲ نمره) در مدار مقابل، وقتی پالس ساعت (clock) از یک به صفر است تغییر میکند، خروجی چه مقدار است؟

فرض میکنیم $Q=1$ پس $\bar{Q}=0$ و فقط حاصل nand قبل از $\bar{Q}$ زمانی صفر میشود که دو ورودی اش برابر ۱ باشند پس ورودی nand قبلی هم باید برابر ۱ شود. دو ورودی آن ۱ هستند پس ورودی k تنها میتواند برابر صفر باشد. از طرفی چون یکی از ورودی های nand Q برابر صفر است پس ورودی دیگر که حاصل nand قبلی است میتواند هر مقداری باشد پس j میتواند هر مقداری باشد. وقتی ساعت صفر میشود: با فرض اینکه مقدار j و k ثابت است پس تفاوتی در خروجی ها ایجاد نمیشود. اگر $Q=0$ فرض میشد باز هم به همین نتیجه میرسیدیم.

ب (۲ نمره) در جدول زیر، وضعیت به ازای ورودی ها وقتی پالس ساعت یک است را بنویسید

در حالت اول تنها میتوان گفت مقدار Q تغییر نمیکند و هرچه باشند همان میماند.

در حالت آخر نیز نمیتوان مقدار Q را پیشبینی کرد.

J	K	Q
0	0	
0	1	0
1	0	۱
1	1	

ج (۲ نمره) فکر میکنید این مدار به چه دردی میخورد؟ با این مدار میتوان نوعی حافظه ساخت.

سوال ۲- (۵ نمره)

الف (۵ نمره) جدول سمبولهای (Symbol Table) برنامه زیر را تکمیل کنید.

Address	Symbol	
		.ORIG x3000 LEA R0, welcome PUTS LD r2, ten loop LD r1, one ADD r2, r2, #-1 BRzp loop nomore HALT
x3003	loop	
x3006	nomore	
x3007	ten	
x3008	welcome	ten .FILL #10 welcome .STRINGZ "Hello world\n" one .FILL #1 . END
x3015	one	

نام و نام خانوادگی:	شماره دانشجویی:	۳ از ۱۱

ب) (۵.۳) جمله LD r2, ten بصورت دستور ماشین بنویسید

0010 010 000000100

ج) (۵.۶) محتوای حافظه از آدرس x_{3007} تا آدرس x_{3013} را بنویسید.

X3007	x000A
x3008	x0048
x3009	x0065
x300A	x006C
x300B	x006C
x300C	x006F
x300D	x0020
x300E	x0077
x300F	x006F
x3010	x0072
x30011	x006C
x3012	x0064
x3013	x000A

سوال ۳-۵) (نمره ۵)

برنامه زیر بدون هیچ هشدار یا کامپایل می شود ولی در زمان اجرا crash می کند و از اجرا باز می ایستد. توضیح دهید علت crash کردن برنامه چیست.

```
int factorial(int x) { if(x==1)
    return 1; return x *
    factorial(x-1);
}
int main(int argc, char**argv)
{ int n = factorial(0);
  return 0;
}
```

Stack overflow جواب: رخ می دهد این برنامه برای مقادیر کمتر از ۱ درست کار نمی کند .

چون هیچگاه با دادن مقدار صفر شرط $x=1$ برقرار نمیشود پس برنامه پایان نمیابد و تا پر شدن حافظه heap ادامه پیدا میکند.

سوال ایراد دارد

.... 0x3006		MAR	MDR
0x3007	LEA R0, x51	x3050	x0E06
0x3008	LDI R1, x51		
0x3009	ADD R2, R1, R0		
...	STI R2, x52		
...			
0x3050	0x0E06		
0x3051	0x3053		
0x3052	0x3054		
0x3053	0x0005		
0x3054	0x000A		
...			

سوال ۶-۱۸ نمره) می‌خواهیم با استفاده از ساختار داده زیر لیست پیوندی (Linked List) از رشته‌های حرفی (String) می‌سازیم.

```
struct node { char * str;          /* heap-allocated string in this
    node./* struct node * next;    /* next node or NULL if
    none/*
};
```

تابع زیر را طوری تکمیل کنید که نود (گره) جدیدی را تخصیص داده و یک کپی از رشته حرفی S را در این نود به لیست پیوندی که از آدرس S^{tart} شروع می‌شود، اضافه نماید. البته یک رشته حرف تنها در صورتی به لیست اضافه می‌شود که از قبل در لیست وجود نداشته باشد.

می‌توانید فرض کنید که از قبل هیچ رشته حرفی تکراری در لیست وجود ندارد. رشته‌های حرفی هم به هیچ ترتیب خاصی مرتب نشده‌اند. لذا رشته S را در هر جایی از لیست می‌توانید اضافه نمایید. توجه داشته باشید که لیست می‌تواند در ابتدا خالی باشد، در اینصورت باید نود اولیه را ایجاد کرده، S را در آن کپی کنید، و آدرس آن را به عنوان نقطه شروع لیست پیوندی بازگردانید. در صورتی که S در لیست موجود باشد، این تابع باید آدرس اول لیست را بازگرداند.

مثال:

دستور زیر رشته حرفی "xyzyzy" را به لیستی که از آدرس $words$ شروع می‌شود اضافه می‌نماید، در صورتی که این رشته حرفی از قبل در آن نباشد.

```
words = insert("xyzyzy", words);
```

در صورت نیاز می توانید توابع دیگری که برای نوشتن برنامه به شما کمک می کنند نیز اضافه نمایید. ضمناً فرض کنید که همه رشته های حرفی در لیست پیوندی با NULL تمام شده اند (Null terminated). به این ترتیب می توانید از توابع کار با رشته های حرفی زیر نیز که در `<string.h>` تعریف شده اند، استفاده نمایید.

`char *strcpy(char *dest, const char *src)`

Copies the string pointed to, by src to dest. It returns a pointer to the destination string dest.

`int strcmp(const char *str1, const char *str2)`

Compares the string pointed to, by str1 to the string pointed to by str2.

This function return values that are as follows:

- if Return value < 0 then it indicates str1 is less than str2.
- if Return value > 0 then it indicates str2 is less than str1.
- if Return value = 0 then it indicates str1 is equal to str2.

`#include <string.h>`

`/* add a copy of s to the list lst if it is not already present, and return a pointer to the possibly updated list.*/`

```
struct node *insert(char *s, struct node *lst)
{
    struct node *current = lst;
    struct node *previous = lst;
    if (lst == NULL)
    {
        lst = (struct node*)malloc(sizeof(struct node));
        lst->str = (char*)malloc(sizeof(char));
        strcpy(lst->str,s);
        lst->next = NULL;
        return lst;
    }
    while (current != NULL)
    {
        if (strcmp(current->str,s)==0)
        {
            return lst;
        }
        previous = current;
        current = current->next;
    }
    previous->next = (struct node*)malloc(sizeof(struct node));
    previous->next->str = (char*)malloc(sizeof(char));
    strcpy(previous->next->str,s);
    previous->next->next=NULL;
    return previous->next;
}
```

سوال ۷-۳۲) (نمره ۳۲) پیش از این با مفهوم درختهای جستجوی باینری (Binary Search Tree- BST) آشنا شده‌اید. BST یک درخت باینری است که نودها (گره‌ها) طوری مرتب شده‌اند که تمام مقادیر در زیر درخت چپ یک نود از مقدار آن نود کوچکترند و همه مقادیر زیر درخت راست از مقدار خود نود بزرگترند. فرض کنید در این سوال درخت جستجوی باینری داریم که مقادیر نودهای آن رشته‌های حرفی هستند. به صورت دقیقتر، مقادیر هر نود پوینترهایی (اشاره گرهایی) به رشته‌های حرفی Null terminated هستند. هر نود توسط ساختار داده زیر تعریف می‌شود:

```
struct tnode {
    /* node for a BST of strings */ char
    *val; /* string value stored in this node */
    struct tnode * left; /* left subtree or NULL if empty */
    struct tnode * right; /* right subtree or NULL if empty */
};
```

راهنمایی: در این سوال می‌توانید از تابع کتابخانه‌ای زیر استفاده نمایید.

```
int strcmp(const char *str1, const char *str2)
```

Compares the string pointed to, by str1 to the string pointed to by str2.

This function return values that are as follows:

- if Return value < 0 then it indicates str1 is less than str2.
- if Return value > 0 then it indicates str2 is less than str1.

- if Return value = 0 then it indicates str1 is equal to str2.

۱۶) (نمره ۱۶) با استفاده از روش بازگشتی (recursive)، تابع زیر را طوری تکمیل کنید که

وقتی رشته حرفی s در درخت جستجوی باینری t وجود داشته باشد مقدار 1 و اگر موجود نباشد مقدار 0 را برگرداند.

```
#include <string.h>
```

```
/* return true (1) if s is contained in the bst */
/* with root t, otherwise return false (0). */
```

```
int contains(struct tnode *t, char *s)
{
    int value;
    if (t == NULL)
    {
        value = 0;
        return value;
    }
    if (strcmp(s, t->val) < 0)
    {
        value = contains(t->left, s);
    }
    else if (strcmp(s, t->val) > 0)
    {
        value = contains(t->right, s);
    }
    else
    {
        value = 1;
    }
}
```

```

    return value;
}
return value;
}

```

ب) (۱۶ نمره) بدون استفاده از روش بازگشتی (recursive)، تابع زیر را طوری تکمیل کنید که وقتی رشته حرفی s در درخت جستجوی باینری t وجود داشته باشد مقدار ۱ و اگر موجود نباشد مقدار ۰ را برگرداند.

```
#include <string.h>
```

```

/* return true (1) if s is contained in the bst */
/* with root t, otherwise return false (0). */ int

```

```

int contains(struct tnode *t, char *s)
{
    int value;
    struct tnode *current = t;
    while (1)
    {
        if (current == NULL)
        {
            value = 0;
            return value;
        }
        if (strcmp(s, t->val) < 0)
        {
            current = current->left;
        }
        else if (strcmp(s, t->val) > 0)
        {
            current = current->right;
        }
        else
        {
            value = 1;
            return value;
        }
    }
}

```

سوال ۸- ۶ (نمره) سابقه فعالسازی Activation record برنامه زبان C زیر را وقتی برای LC-3 کامپایل شده،

در پایان تابع factorial و قبل از بازگشت از آن به تابع main تکمیل نمایید. مقادیری را که شما نمیتوانید بدانید با علامت سوال مشخص نمایید.

محتوا	نوع و نام محتوا	آدرس
0	Int result	x2981
x2986	Pointer dynamic link for factorial(1)	x2982
1	Return address pointer for factorial(1)	x2983
?	Return int value	x2984

```

int factorial(int num); void
main ()
{
    int num=3;
    num = factorial (num);
}

int factorial(int num)

```

<pre> { int result=0; if(num==1) return 1; result = num*factorial(num-1); return result; } </pre>	x2985	Int num	1
	x2986	Int result	0
	x2987	Pointer dynamic link for factorial(2)	X2991
	x2988	Return address pointer for factorial(2)	2
	x2989	Return int value	?
	x2990	Int num	2
	x2991	Int result	0
	x2993	Pointer dynamic link for factorial(3)	X2997
	x2994	Return address pointer for factorial(3)	?
	x2995	Return int value	6
	x2996	Int num	3
	x2997	Int num	3
	x2998	Pointer dynamic link	?
	x3000	Return address pointer for main	?
	X3001	Return void value	?

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			0	00		SR2		
ADD ⁺	0001				DR			SR1			1	imm5				
AND ⁺	0101				DR			SR1			0	00		SR2		
AND ⁺	0101				DR			SR1			1	imm5				
BR	0000				n	z	p	PCoffset9								
JMP	1100				000			BaseR			000000					
JSR	0100				1	PCoffset11										
JSRR	0100				0	00		BaseR			000000					
LD ⁺	0010				DR			PCoffset9								
LDI ⁺	1010				DR			PCoffset9								
LDR ⁺	0110				DR			BaseR			offset6					
LEA ⁺	1110				DR			PCoffset9								
NOT ⁺	1001				DR			SR			111111					
RET	1100				000			111			000000					
RTI	1000				000000000000											
ST	0011				SR			PCoffset9								
STI	1011				SR			PCoffset9								
STR	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								
reserved	1101															

Figure A.2 Format of the entire LC-3 instruction set. Note: + indicates instructions that modify condition codes